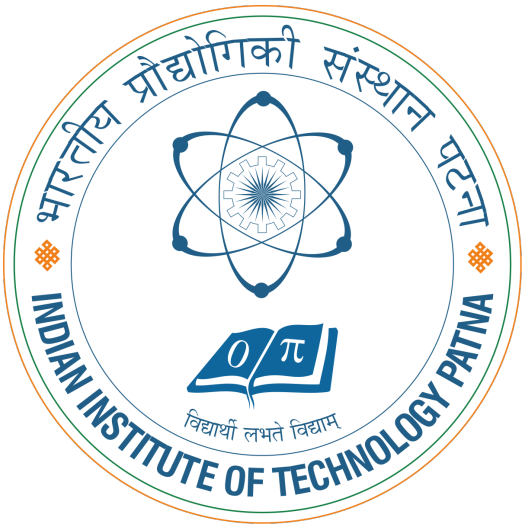
Sequential Handwritten Digit Classification

(Row-by-Row Sequential Modeling)



Ritik Yadav (2621MA03)

Sahnil Siddique (2621MA02)

Suman Acharya (2621MA11)

Department of Mathematics

Indian Institute of Technology Patna

1. Introduction

This task converts each 28 × 28 handwritten Kannada digit image into a sequence of 28 feature vectors. At time step t, the model receives one image row containing 28 pixel values. A recurrent neural network processes the row sequence and predicts one of ten digit classes.

2. Objectives

- **Pixel Normalization:** Normalize grayscale pixel intensities from the original range $[0, 255]$ to the interval $[0, 1]$ using

$$x_{\text{norm}} = \frac{x}{255.0}.$$

- **Sequential Representation:** Represent each 28×28 grayscale image as a sequence of row vectors:

$$X = [x_1, x_2, \ldots, x_{28}]^T \in \mathbb{R}^{28\times 28},$$

  where each row vector is

$$x_t \in \mathbb{R}^{28}, \qquad t = 1, 2, \ldots, 28.$$

  Thus, each image is processed over $T = 28$ time steps, with 28 features at each time step.

- **Model Training and Comparison:** Train and compare the following recurrent neural network architectures:
  - Vanilla Recurrent Neural Network (RNN)
  - Long Short-Term Memory (LSTM)
  - Gated Recurrent Unit (GRU)

- **Performance Evaluation:** Evaluate the classification performance using training accuracy, test accuracy, training-loss curves, and confusion matrices. The classification accuracy is calculated as

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \times 100\%.$$

3. Data preprocessing and representation

For each pixel x, normalization is performed using $x_norm = x/255.0$. Thus, every image has shape (28, 28), and the recurrent model sees 28 time steps with 28 features at each step. The labels correspond to the ten classes 0 through 9.

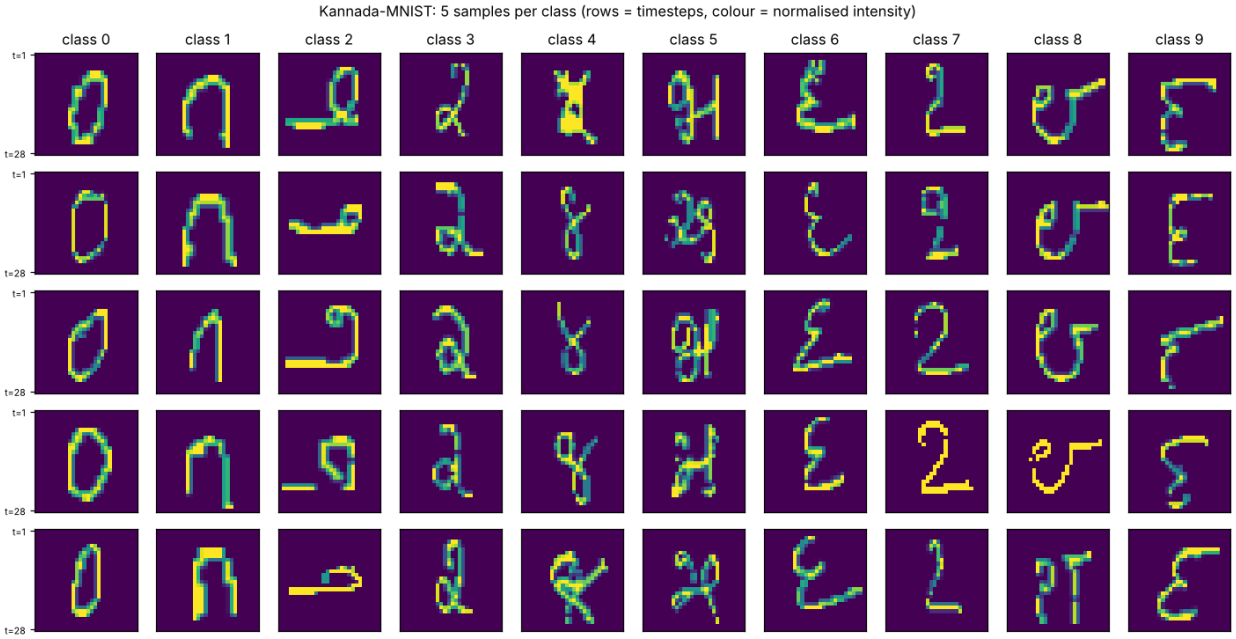


Figure 1: The sample grid illustrates variation in handwritten shapes across the ten digit classes.

4. Models

Vanilla RNN: updates a hidden state at each row and uses the final representation for classification. It is simple but may struggle to preserve information over longer sequences.

LSTM: uses input, forget and output gates with a memory cell to control information flow across time steps.

GRU: uses update and reset gates, offering a gated recurrent design with fewer gates than LSTM.

5. Training behaviour and hyperparameter comparison

The notebook includes training curves for multiple configurations and a comparison of training/test accuracy. Lower loss over epochs generally indicates that the optimizer is reducing prediction error; validation/test performance is needed to assess generalization.

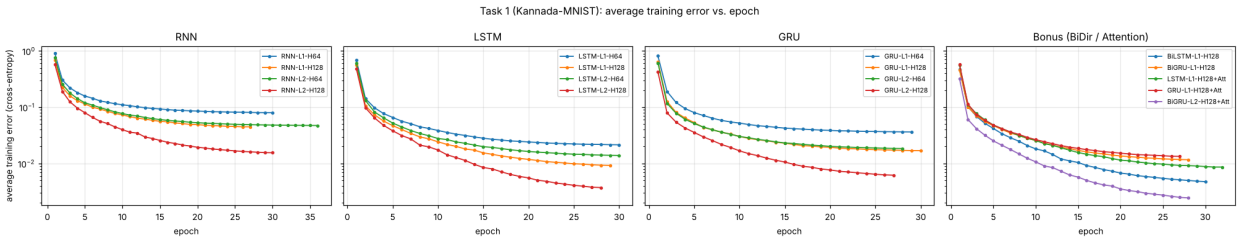


Figure 2: Curves compare loss trends over epochs. The plot should be interpreted together with held-out accuracy, not in isolation.

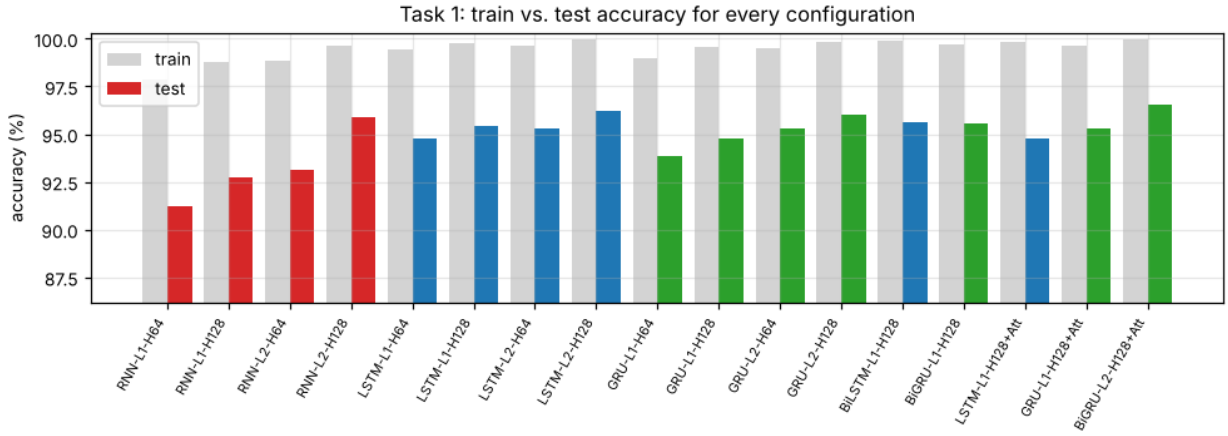


Figure 3: The bars summarize the accuracy values displayed by the notebook for the tested configurations.

6. Confusion-matrix analysis

A confusion matrix compares true labels (rows) with predicted labels (columns). Large values on the main diagonal represent correctly classified examples; off-diagonal values indicate which classes are confused. These matrices help identify class-specific errors that overall accuracy can hide.

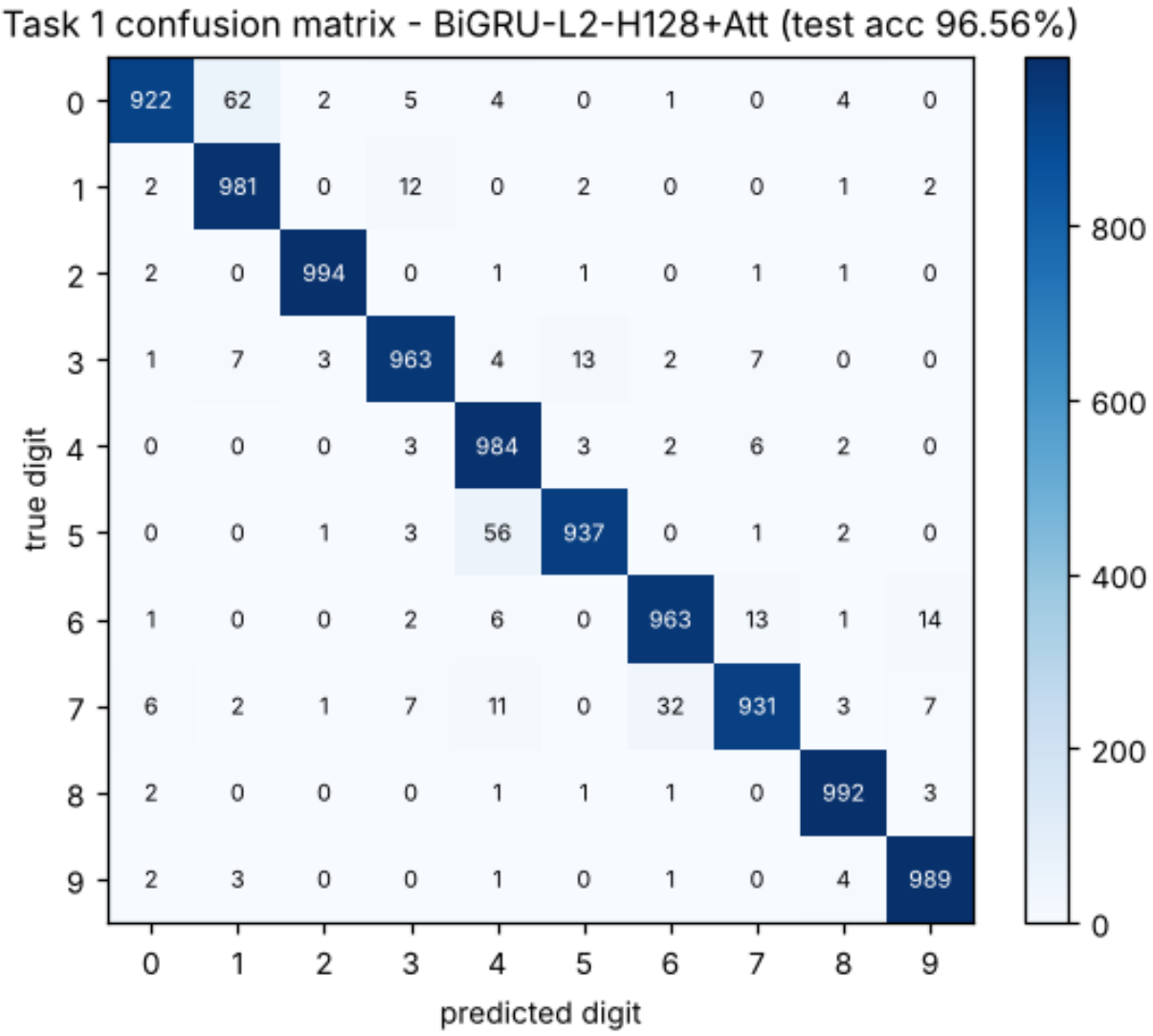


Figure 4: GRU confusion matrix

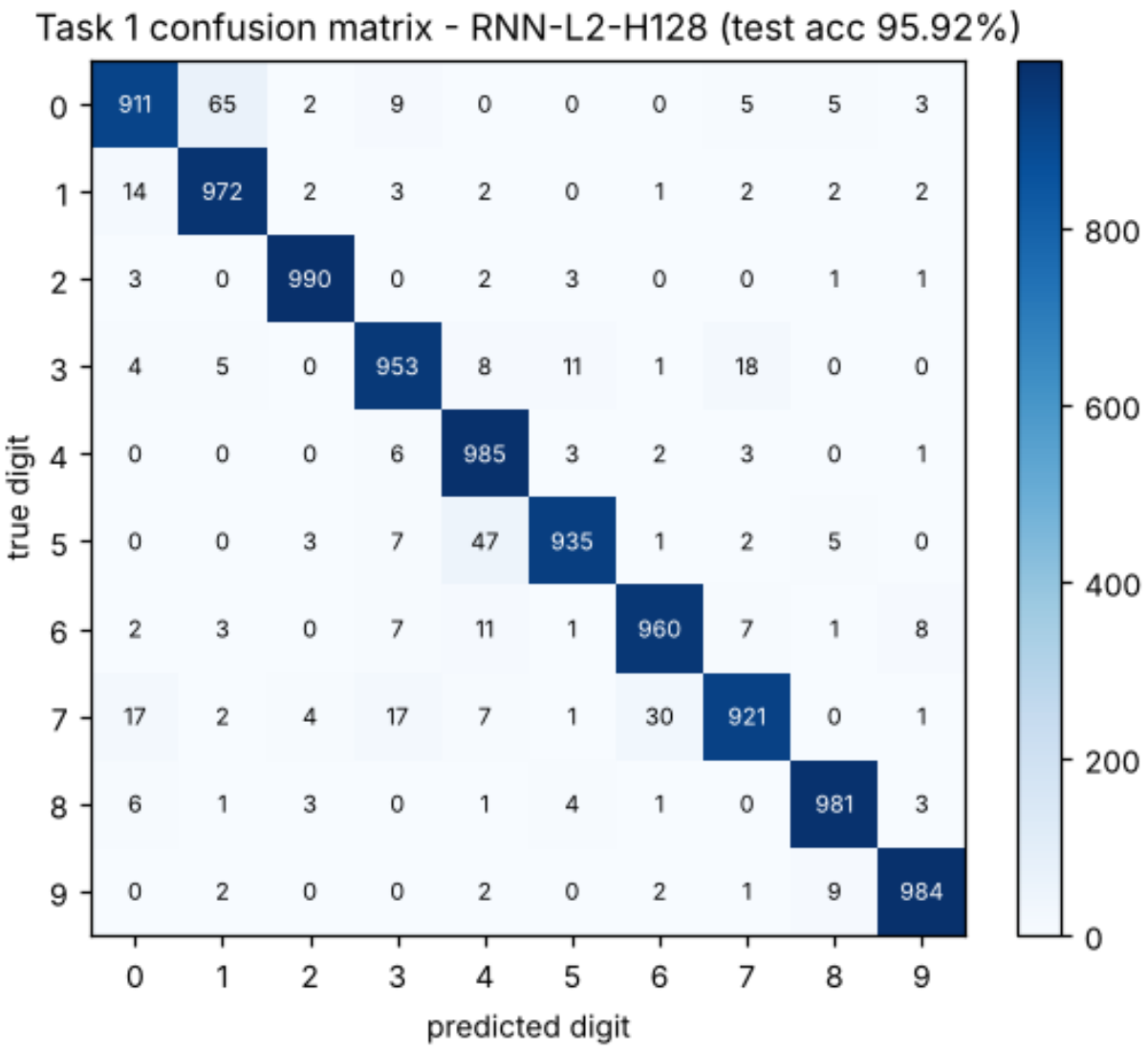


Figure 5: Vanilla RNN confusion matrix

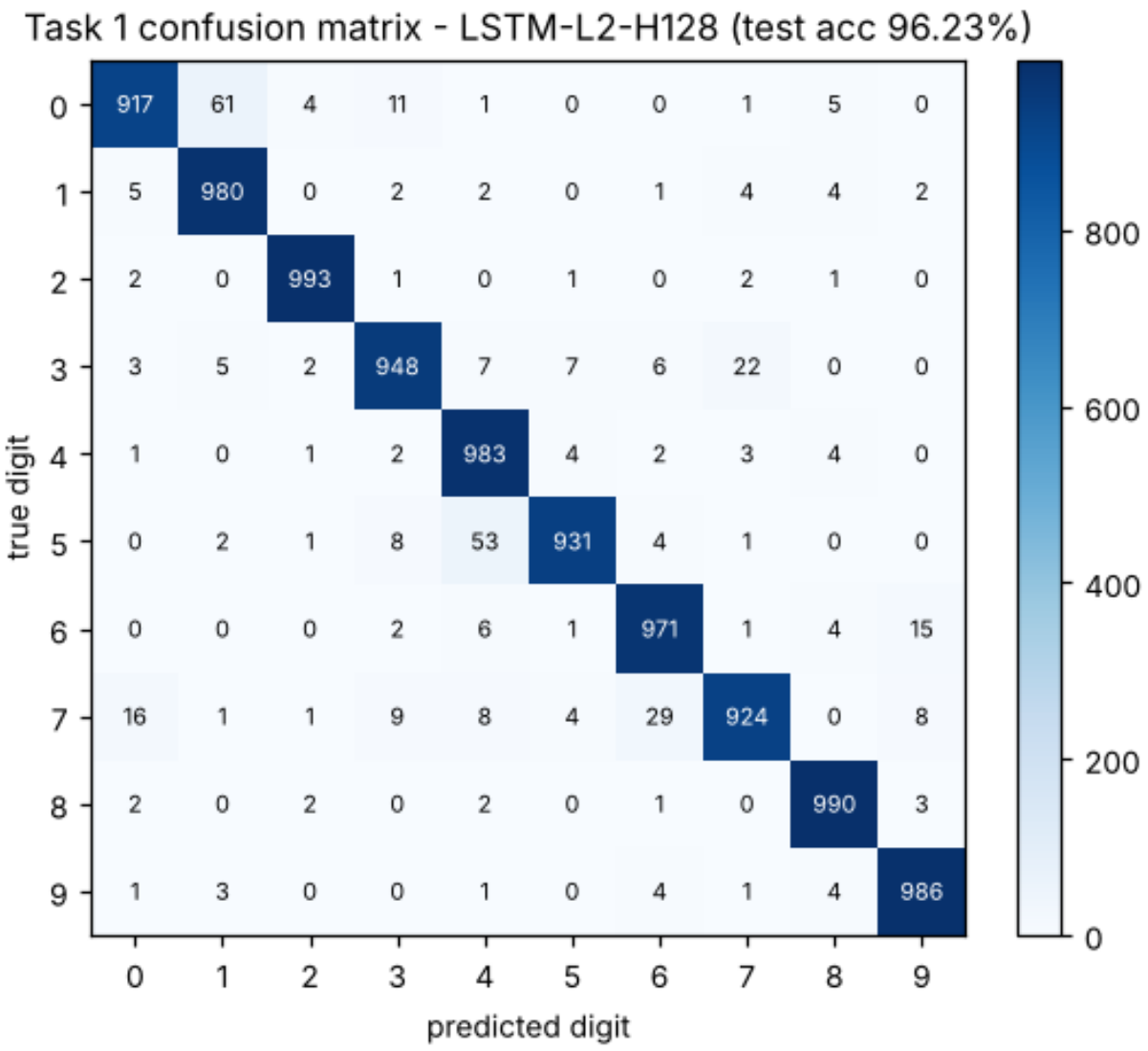


Figure 6: LSTM confusion matrix

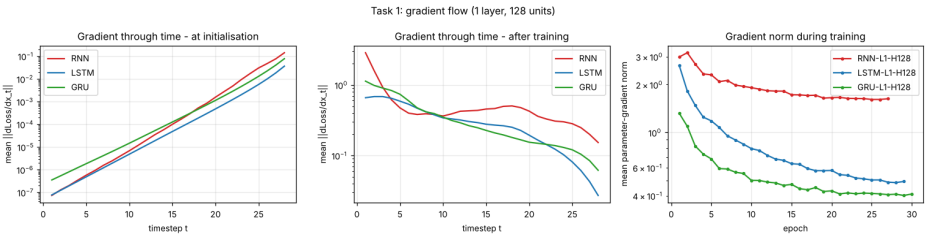


Figure 7: Gradient diagnostics can help reveal whether gradients become very small or large during training; they should be read alongside loss and accuracy.

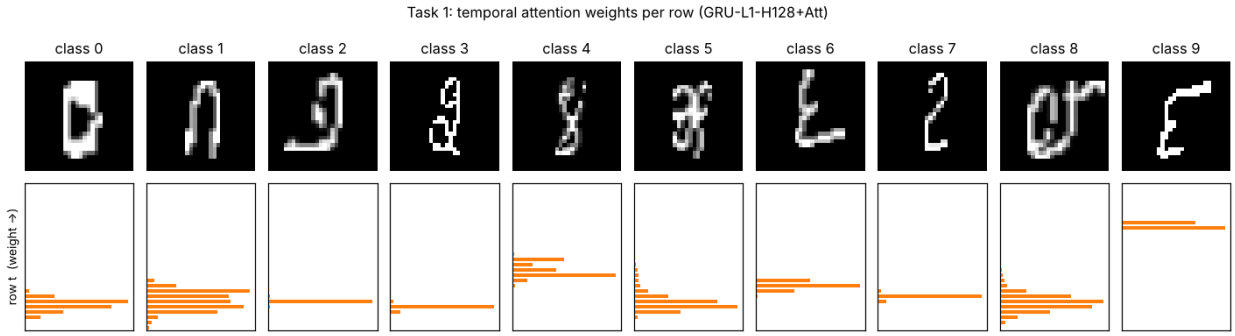


Figure 8: This figure shows sample digit inputs alongside time-step-dependent values produced by the notebook visualization. It is a qualitative aid, not a substitute for quantitative evaluation.

7. Additional visualizations

The following plots provide supporting evidence about learning dynamics and how the model responds across digit classes.

8. Conclusion

The assignment demonstrates how a static image can be treated as sequential data by feeding its rows to a recurrent model. RNN, LSTM and GRU architectures can then be compared using learning curves, accuracy summaries and confusion matrices. A sound final comparison should consider test accuracy, class-wise errors, convergence behaviour and model complexity together.